

Rime Voice AI — Deep Technical Research & Hackathon Build Brief (DataForge × Pathway × Rime)

TL;DR

Build a telephony-first, code-switched Indian-English/Hindi ("Hinglish") voice agent that Rime uniquely enables via Coda (the only current Rime cloud model with Hindi, `lang: hi`, 2 Hindi voices), and pair it with a rigorous latency + interruption + pronunciation eval harness — this hits every rubric axis (voice necessity, hard engineering, Rime integration, reproducibility) and avoids everything already in Rime's catalog (which is English/Spanish and document-reading focused).

Rime's current flagship is Coda (`modelId: coda`), **released May 2026**, not Arcana (cloud Arcana sunset to Coda on Aug 15, 2026) or Mist v2. Coda = quality + 9 languages + sub-100ms engine latency; Mist v3 = fastest (37ms P50 TTFA) but no Hindi and no inline pronunciation; Mist v2 = the only model with inline phoneme control. Query the **live catalog** at `https://users.rime.ai/data/voices/all-v2.json` (public, no auth) rather than hardcoding speakers.

Use LiveKit Agents (Python `livekit-agents[rime]~1.5`) for orchestration — it gives you adaptive barge-in, semantic turn detection, SIP/Twilio telephony, and a first-party Rime plugin with WebSocket streaming and word-level timestamps. The hard, differentiating engineering is code-switch handling, telephony μ -law degradation, and reproducible eval — not the happy-path demo.

Key Findings

Rime model lineup (current, Sept 2026)

Coda (`modelId: coda`) — flagship, released May 2026. LLM backbone + speech engine trained on full-duplex conversational data. [Rime](#)

↗
Highest human-eval quality scores. **Sub-100ms model latency** [Rime Docs](#)

↗
on GPU engine (self-host/on-prem); cloud adds 25–50ms US network RTT. Word-level timestamps. Supports `spell()`. **Does NOT support inline pronunciation control or SSML**. Native sample rate 24kHz. **Note a live-documentation version conflict:** the currently-rendered `docs.rime.ai/docs/models` and `/docs/voices` pages state **253 voices across 9 languages** (English, Arabic, French, German, Hindi, Italian, Japanese, Portuguese, Spanish; 162 English, 2 Hindi), while a cached/older snapshot of the same models page states **"Coda serves 8 languages across 184 voices... 121 English, only 2 Hindi"** (no Italian). **Both agree Hindi = 2 voices**. Treat the live catalog endpoint as ground truth at build time.

Mist v3 (`modelId: mistv3`) — released March 2026, lowest latency. **TTFA ~37ms P50 / 56ms P90** in Rime's benchmark. 78 voices. English/French/German/Spanish only (**no Hindi**). [rime](#)

➤ Supports custom pauses; **no inline pronunciation control**. `speedAlpha` >1.0 = faster. Native sample rate 22.05kHz. **This is the model served if you omit** `modelId` — always set `modelId` explicitly. [Rime Docs](#)

➤ **Mist v2** (`modelId: mistv2`) — released Feb 2025. **Only model with inline pronunciation control** [Rime Docs](#)
(`phonemizeBetweenBrackets`). 138 voices. EN/FR/DE/ES. ~175ms median on-prem (A10G, 40–50 char sentences).
Quirk: `speedAlpha` <1.0 = faster (inverted vs every other model). [LiveKit](#)

➤ **Mist (legacy, `mist`)** and **v1** — deprecated.

Arcana — being retired on cloud; Arcana requests switch to Coda Aug 15, 2026. On-prem Arcana images remain. [Rime Docs](#)

➤ Do NOT build a new cloud project on Arcana.

API surface (all primary, docs.rime.ai)

Auth: bearer token — `Authorization: Bearer $RIME_API_KEY`. [rime](#)

➤ Create key in Rime dashboard (API Tokens page).

HTTP synthesis (REST/streaming): `POST https://users.rime.ai/v1/rime-tts`. Headers: `Authorization`, `Content-Type: application/json`, `Accept: audio/mpeg` [rime](#)

➤ (mp3) / `audio/wav` / `audio/pcm` / `audio/L16` / `audio/mulaw` / `audio/ogg`. Body: `{"speaker", "text", "modelId", "lang", ...}`. Stream by iterating response chunks.

WebSocket `/ws` (Mist v2 plain-text): `wss://users-ws.rime.ai/ws?speaker=...&modelId=mistv2&audioFormat=mp3`.
Send bare text tokens; receive raw audio bytes. [Rime Docs](#)

➤ Commands: `<CLEAR>` (clear buffer on interruption), `<FLUSH>` (force synth), `<EOS>` (synth + close). Buffers to punctuation `. , ? !`. **1,000 character limit per request.** [Rime Docs](#)

➤ **WebSocket JSON** `/ws3` (Coda) and `/ws2` (Mist v2 JSON) — JSON API with word-level timestamps, context IDs, interruption support. Base for LiveKit/Pipecat plugins. (Pipecat's `RimeTTSService` historically pointed at `/ws2` and `/ws3`; the current LiveKit plugin default WS base is `wss://users-ws.rime.ai`.)

Metadata (public, no auth): `GET https://users.rime.ai/data/voices/all-v2.json` (voice names keyed by `modelId`→language) and `GET https://users.rime.ai/data/voices/voice_details.json` [rime](#)

➤ (demographics: gender, age, dialect, lang). **This is the "current catalog" the challenge requires — query at startup, don't hardcode.** Note: a Nov 2025 changelog renamed `model_id`→`modelId`, `name`→`speaker`, `region`→`dialect` in `voice_details.json`.

Utility: `POST https://users.rime.ai/oov` (out-of-vocabulary/coverage check) and `POST https://optimize.rime.ai/textnorm` (text normalization preview — POST tricky strings to see exactly how Rime will read them).

Request parameters (exact names/types)

`speaker` (str, required) — must be valid for chosen model+lang; verify via catalog endpoint.

`text` (str, required) — ≤1,000 chars/request.

`modelId` (str) — `coda` / `mistv3` / `mistv2` / `mist`.

`lang` (str) — BCP-47 tag (`en`, `hi`, `es`, `fr`, `de`, `ar`, `it`, `ja`, `pt`); legacy 3-letter ISO 639-2 (`eng`, `hin`) still accepted. [Rime](#)



`samplingRate` (int) — 4000–44100, default 22050. Telephony: request 8000 directly.

`speedAlpha` (float, default 1.0) — Coda/Mist v3: >1.0 faster; Mist v2: <1.0 faster (inverted).

`time_scale_factor` / `timeScaleFactor` (float) — Coda & Mist v3 only, HTTP only (ignored on WS). [LiveKit](#)



`inlineSpeedAlpha` (str) — comma-separated per-word speed for `[bracketed]` words.

`pauseBetweenBrackets` (bool) — `<200>`-style ms pauses (Mist family).

`phonemizeBetweenBrackets` (bool) — `{h'El.o}` custom phonemes. **Mist v2/v1 only.**

`noTextNormalization` (bool) — Mist/Mist v2 only; skips normalization to cut latency (risks mispronouncing digits/abbrev). [LiveKit](#)



`segment` (str) — `bySentence` (default) / `immediate` [LiveKit](#)



/ `never` (WS text segmentation).

`spell()` — inline function to read IDs char-by-char (Mist family processes it; Coda passes it through to the model unprocessed).

Latency (published, primary)

Rime benchmark (single Lambda H100 SXM5, driver 595.58.03): Coda TTFA 96ms P50 / 98ms P90 @ 1 concurrency, 150ms P50 / 181ms P90 @ 12 concurrency; RTF P99 0.33. Mist v3 TTFA 37ms P50 / 56ms P90 (flat across concurrency); RTF P99 0.004.

Cloud API: sub-200ms end-to-end typical; ~150–200ms depending on region. [Rime](#)



Historical blog cites ~175ms TTFB, sub-100ms enterprise. Cerebrum self-host claims 80ms TTFB vs ~300ms cloud.

Network: US East↔West coast-to-coast ~60–85ms floor; same-region 1–10ms. **No India/APAC regional endpoint is documented** — a Chennai user hitting US endpoints will add substantial RTT. Real constraint to measure and design around.

Human turn-taking gap averages ~200ms; target total STT→LLM→TTS pipeline <700ms.

Pricing / free tier

Usage-based, per Rime's pricing page (rime.ai/pricing): **"Starter begins at \$0.03 per 1,000 characters (about a minute of audio): \$0.03 for Mist and \$0.05 for Coda. Enterprise plans move to volume pricing."** (Arcana historically \$0.04.)

Free onboarding: every new account starts with 3,000 free minutes on the Starter plan (independent research corroborates: "\$0.03 for Mist... \$0.05 for Coda per 1,000 characters — with 3,000 free minutes on the Starter plan"). Ample for a hackathon. Older tier structure: Starter \$5/100k chars, Developer \$19–25/500k, Pro \$99/3M, Business \$249/10M.

From docs.rime.ai Integrations + guides:

OpenClaw document-reader Telegram bot — reads docs aloud / spoken summary / two-voice podcast as Telegram voice notes. [rime](#)

↗

Its `rime.py`: normalizes whitespace and splits into ~400-char sentence-aligned chunks (`CHUNK_SIZE`); POSTs each chunk to `/v1/rime-tts` (model `coda`, `Accept: audio/L16`, with `speaker`/`samplingRate`/`speedAlpha`) for raw PCM; concatenates chunks into one buffer with ~0.3s silence between them (per-segment voices in podcast mode); then shells to `ffmpeg` to encode PCM→OGG Opus (libopus, 64k VBR, `-application voip`) for Telegram voice notes.

Lovable integration — Rime voices in Lovable-built web apps.

Replit integration — Rime in Replit IDE projects.

SignalWire — Rime voices in SWML telephony/IVR agents (`rime.<speaker>:<model>`).

Together AI — Rime TTS co-located with LLM/STT on dedicated endpoints.

Vapi — Rime as TTS for Vapi voice agents (`voice.provider = rime-ai`).

VideoSDK — Rime with VideoSDK AI Agent SDK (RTC).

LiveKit — real-time voice agent tutorial + starters.

Pipecat — Rime in Pipecat pipelines.

Daily — Rime with Daily transport.

Quickstarts: "TTS in five minutes" (HTTP), CLI quickstart, "Build a voice agent"

(Next.js/Vite/Express/Node/FastAPI WebSocket starters). [rime](#)

↗

On-prem quickstart (Docker Compose/K8s; Arcana images at `us-docker.pkg.dev/rime-labs/...`).

MCP server (Claude/Codex), streaming/WebSocket guides.

Enterprise voice cloning; Speech QA guide; text normalization, prompting, pronunciation guides.

The catalog is overwhelmingly **English/Spanish, document-reading, and integration-glue**. There is **no Indian-language, code-switched, or telephony-adverse-conditions flagship project** — that's the open lane. (Also note a related but separate `livekit-examples/rime-multilingual-demo` on GitHub demonstrates language switching with LiveKit + Rime — study it, but don't reproduce it.)

"Writing for the ear" / Prompting guide (Brooke Larson, Rime co-founder) — concrete techniques

Text is a lossy, low-dimensional representation of speech; write like people talk.

Contractions; start sentences with And/But/So; drop formal connectors (furthermore/additionally).

Light disfluencies written into text (um, uh, yeah, well, I mean, you know) — "sprinkle, don't stack" (two ums = a bug). Don't use tags; the disfluency IS the prompt.

Punctuation = prosody: comma (short pause/slight rise), period (falling pitch), question mark (rising), ellipsis (hesitant/trailing, sparingly), semicolon (between comma and period).

Sentences <25 words (ideally <15) or they sound breathless.

Show don't tell: give examples ("Yeah, I can help with that. One sec.") not adjectives ("friendly"). Describe personality as observable speech patterns ("starts sentences with 'yeah'").

Coda **does NOT accept SSML** — no `<break>`/`<emotion>`; only `spell()` is supported inline.

Normalize cleanly: pass currency/dates/times/phones through unchanged; expand dates-without-year, MM/YYYY, bare hours, decades, Q1/1H, non-dollar currency shorthand; use `spell()` for IDs/codes/SKUs but NOT standard phone numbers; avoid dashes in numbers (cause weird pauses). The **full drop-in system prompt is published** in the docs (4 parts: sound-like-a-person, normalize, `spell()` for IDs, invariants) — paste it verbatim into your LLM system message and adapt the persona.

LiveKit + Rime (orchestration)

Plugin: Python `uv add "livekit-agents[rime]~1.5"`; Node `@livekit/agents-plugin-rime@1.x`. Or LiveKit Inference (`rime/coda`, no separate Rime key; billing via LiveKit Cloud).

Usage: `rime.TTS(model="coda", speaker="celeste", speed_alpha=0.9)`; enable `use_websocket=True` for streaming + word-level timestamps (TTS-aligned transcriptions); `segment="bySentence"|"immediate"|"never"`. String descriptor shortcut: `tts="rime/coda:celeste"`.

Coda ignores `reduce_latency`, `pause_between_brackets`, `phonemize_between_brackets`, `temperature`, `top_p`, `repetition_penalty`. Default Coda speaker = `lyra`. Plugin `audio_format` valid values `pcm`/`mp3`, default `sample_rate` 16000.

Interruption/barge-in: LiveKit runtime cancels active TTS on VAD-detected user speech, rolls back the interrupted LLM turn, restarts from STT. **Adaptive Interruption Handling** (default on Cloud, Python Agents v1.5.0+/TS v1.2.0+) uses a CNN + audio-encoder model to distinguish true barge-ins from backchannels ("uh-huh"). Per LiveKit's launch, it **"rejected 51% of barge-ins that would have been falsely triggered by traditional voice activity detection and detected true interruptions faster in 64% of cases,"** with a 1.0s start cooldown. `resume_false_interruption` + `false_interruption_timeout` resume after false positives (cough/noise). Config via `turn_handling` / `interruption.mode` (`adaptive` default on Cloud, else `vad`). `discard_audio_if_uninterruptible` default True.

Turn detection: ~135M SmolLM-v2 fine-tune (semantic end-of-turn) + Silero VAD. v1 full model = Cloud only; v1-mini runs locally on CPU. Audio turn detector resamples to 16kHz internally so **8kHz telephony works**. Tune VAD silence per vertical (IVR ~250ms, sales ~400ms, healthcare ~600ms).

Telephony: LiveKit SIP + Twilio integration (create inbound SIP trunk via LiveKit CLI script). μ -law 8kHz path.

Multilingual / code-switching — the India angle

Coda supports Hindi (`lang: hi`), **2 Hindi voices** on cloud (2 of 253/184). Mist v3 has NO Hindi. Arcana historically had Hindi (voices added Oct 2025) but is sunsetting.

Rime markets code-switching, but the **documented/proven code-switch is English↔Spanish↔Spanglish** ("just like a native speaker would"). Arcana training explicitly encodes "multi-lingual code-switching" and accents/idiolects, [Rime](#)

↗

but **Hindi-English/Hinglish code-switching is NOT a documented, tested capability** — this is exactly the unverified frontier to probe and measure. Expect failure modes: English pronunciation of Hindi loanwords, prosody breaks at switch points, and the architectural fact that a Coda voice serves ONE language (no single voice crosses EN↔HI), [rime](#)

↗

so mid-utterance single-voice code-switch is uncertain.

Indian-accented English is not a documented Coda voice category; featured Coda voices are American/Australian/English. Genuine gap.

Competitive/differentiation intel

Prior Rime hackathon (AssemblyAI + LiveKit + Rime + Accel, SF, Sep 19 2025): winners included Pathfinder (career-discovery via dialogue), Scam Fighters (kept a scammer engaged 33 min, deepfake detection), True Voice (brand-voice matching from website analysis). Stack: AssemblyAI STT + LiveKit + Rime TTS. Judges rewarded a *clear, voice-necessary problem + a measurable outcome*.

DataForge 2026: organized by KDAG, IIT Kharagpur; sponsored by Pathway (Dragon Hatchling/BDH post-transformer architecture) and Rime. **Rime raised \$24M Series A led by M13 (with Twilio Ventures, Corazon Capital, Unusual Ventures), announced July 15, 2026** (Business Wire); Rime "powers nearly 100 million phone calls monthly for global leaders like Mayo Clinic, Dialpad, Upstart, and Asurion." Prize pool ₹3,00,000 (₹1.5L / ₹1L / ₹0.5L). Teams ≤4.

Judging rubric: 25% problem+necessity of voice, 25% hard voice engineering, 20% Rime integration, 20% evidence/reproducibility, 10% demo clarity. **40% of the score is engineering + reproducibility — a benchmark/eval harness is disproportionately rewarded.**

Eval tooling: Coval (continuous independent TTS TTFA benchmark, refreshes ~30 min; includes Rime Mist-v3 and Arcana); Gadium published TTFA methodology (P25-P95, WebSocket for all providers); Hamming AI (interruption-handling runbook); LiveKit's own test framework + agent simulations (beta). WER/MOS/intelligibility are the standard TTS metrics.

Realtime alternatives (context)

Pipecat (`[pipecat-ai]`) — has Rime plugins: `[RimeTTSService]` (WS JSON, interruptions, context, word-timing), `[RimeHttpTTSService]`, `[RimeNonJsonTTSService]`. Good for fully custom pipelines and fine-grained control.

Vapi — hosted voice-agent platform; Rime via `[voice.provider = rime-ai]`. [Vapi](#)

Fastest to a phone number, least control — likely too "off-the-shelf" for a 25%-engineering rubric.

Qwen Audio Agent (`[QwenAudio/qwen-audio-agent]`) — open-source realtime full-duplex voice runtime; DashScope `[qwen-audio-3.0-realtime-plus]` frontend + backend agent, with a floating desktop voice orb and background task delegation. Underlying **Qwen3-Omni** (Thinker-Talker MoE, native speech-to-speech, 10 output languages) reports a **theoretical end-to-end first-packet latency of 234ms** (cold-start, 30B-A3B, concurrency 1) per its technical report (arXiv:2509.17765); a Salesforce tutorial measured ~702ms audio-to-audio over the DashScope Realtime API and found no fully self-hostable end-to-end path (local vLLM runs the Thinker but not the Talker). **Relevant only as a native-S2S baseline to beat — using it instead of Rime would defeat the challenge**, so treat it as a comparison point, not the build.

Details

Why a telephony code-switched Indian-language agent is the strongest submission

The rubric rewards (a) a problem where voice is *necessary*, (b) *hard* engineering, (c) deep *Rime* integration, (d) *reproducible evidence*. India-facing phone use cases (rural service access, kirana/SME ordering,

appointment/logistics booking, government-scheme helplines) are genuinely voice-native (low literacy, no app friction, phone-first), force code-switching, and exercise Rime's least-tested surface (Hindi + Indian English + μ -law 8kHz + high India $\rightarrow$ US network RTT). No catalog item touches this. The difficulty is real and measurable, and the eval harness supplies the reproducibility points.

Concrete acceptance tests to build (this is the 40% engineering+reproducibility score)

Time-to-first-audio: instrument STT-final $\rightarrow$ LLM-first-token $\rightarrow$ TTS-first-byte $\rightarrow$ speaker. Report P50/P90/P99 over ≥ 100 turns. Target end-to-end $< 700\text{ms}$; show the India $\rightarrow$ US-endpoint RTT penalty and a mitigation (prewarmed/pooled connections and keep-alive — cold connections add 50–150ms; `segment="immediate"`; sentence-boundary chunking; filler-word priming; short < 500 -token system prompt).

Interruption correctness: scripted barge-in vs backchannel test set; measure false-interruption rate and time-to-stop. Use LiveKit adaptive interruption + Rime `<CLEAR>`; verify stale TTS is fenced (nothing the user "heard" is re-spoken; reconcile heard-vs-generated via word timestamps).

Pronunciation: fixed test list of Indian names, place names, PIN codes, phone numbers, ₹ amounts, and domain terms; score human intelligibility (MOS) and machine WER via re-transcription. Compare Coda default vs `spell()` vs `/textnorm` preprocessing. Coda has no inline phoneme control — quantify where it fails and whether Mist v2's `phonemizeBetweenBrackets` recovers it (at the cost of Hindi, which Mist lacks — document that trade-off).

Code-switch quality: Hinglish/Tanglish test utterances; measure switch-point prosody and mispronunciation. This is where you'll find and document real Rime failure modes — **negative results here are still strong evidence** and are non-duplicative.

Telephony degradation: run the same suite over browser mic (24kHz PCM) vs μ -law 8kHz telephony; quantify intelligibility loss and what breaks (sibilance, digit confusion).

Telephony path specifics

Request 8kHz μ -law directly (`audioFormat=mulaw`, `samplingRate=8000`) to avoid client resampling. Twilio Media Streams / SIP trunk into LiveKit. Going from browser mic to real phone audio breaks: echo cancellation (must be client/device side; use Krisp's telephony-tuned model in LiveKit), VAD thresholds (narrowband), digit/number intelligibility, and sibilant consonants. LiveKit's turn detector resamples 8kHz $\rightarrow$ 16kHz internally so it still functions on PSTN.

Recommendations

Stage 1 — Validate the frontier (day 1): Before committing, empirically test Rime Coda Hindi + Hinglish with the 2 Hindi voices and Indian-English strings via `POST /v1/rime-tts` and the `/textnorm` endpoint. Query `all-v2.json` and `voice_details.json` to get the exact current Hindi speaker names (filter `modelId: coda`, `lang: hin`/`Hindi`). **Decision threshold:** if Coda Hindi/Hinglish quality is usable $\rightarrow$ build the code-switched India agent. If it's poor $\rightarrow$ pivot the thesis to *"measuring and mitigating Rime's code-switch/pronunciation failure modes for Indian users"* — a benchmark submission that is still rubric-strong (40% engineering+reproducibility) and non-duplicative. Either way you have a submission.

Stage 2 — Orchestration (day 1-2): LiveKit Agents (Python) + Rime plugin with `use_websocket=True`, `model="coda"`, adaptive interruption on. STT: Deepgram or AssemblyAI (proven in prior Rime hackathon; both handle Indian English reasonably). LLM: a fast model, system prompt <500 tokens. Wire Twilio SIP for the phone path.

Stage 3 — Apply Rime's prompting guide: Use the published drop-in system prompt; write disfluencies/punctuation for prosody; `spell()` for codes/PINs; `/textnorm` for every ₹/date/number string. Query the live catalog at startup — this **explicitly satisfies the "use the current catalog, don't hardcode" requirement**, so log it and show it in the demo.

Stage 4 — Build the eval harness (day 2-3, highest ROI): Implement the 5 acceptance tests with reproducible scripts, fixed test sets, and P50/P90/P99 tables + MOS/WER. This alone captures the 20% reproducibility + much of the 25% engineering. Cite Coval/Gradium methodology for credibility.

Stage 5 — Demo (day 3): Live phone call in Hinglish, on-screen latency/interruption telemetry, and a one-page results table. Keep it <3 min; lead with the voice-necessity problem.

Benchmarks that change the plan: end-to-end P90 >900ms → self-host Coda (Cerebrum/Together/on-prem) or move your orchestrator to a US region and cache aggressively; Hindi MOS unusable → pivot to the benchmark thesis; interruption false-positive rate >10% → tune VAD/adaptive thresholds per vertical; μ -law digit-WER spike → add `spell()` + confirmation readbacks for numbers.

Caveats

No India/APAC Rime regional endpoint is documented. All published RTT figures are US-metro. Chennai→US latency must be measured firsthand; it may dominate your budget. Consider self-host/on-prem or a US-region deploy of your own orchestrator, and report the measurement either way.

Hindi-English code-switching on Coda is NOT a documented/verified capability — only EN↔ES/Spanglish is. A Coda voice serves one language; [Rime Docs](#)

mid-utterance single-voice code-switch may not work as hoped. Treat this as a research question, not an assumption.

Exact names of the 2 Coda Hindi voices were not verifiable from a Rime primary source during research (the catalog JSON and `/docs/voices-coda` Hindi section were fetch-restricted). Query `voice_details.json` directly. A third-party (Telnyx) list of Rime Hindi voice names includes Telnyx-private voices and should not be trusted as the Coda set.

Voice/language counts drift between doc versions (253 voices/9 langs incl. Italian on the live page vs 184/8 in a cached snapshot; "600+ voices/50+ languages" is whole-lineup marketing spanning legacy models). Both docs versions agree Hindi = 2 voices. Use the live catalog endpoint as ground truth at build time.

Model quirks that will bite: omitting `modelId` serves Mist v3 (no Hindi); `speedAlpha` is inverted on Mist v2; Coda silently ignores many Mist parameters; unsupported speaker+lang pairings may NOT return an error — always validate against the catalog.

Adaptive interruption's "51% false-barge-in rejection / 64% faster" figures are LiveKit's own vendor benchmark; validate on your own Indian-English call traffic.

Prior-hackathon results, funding, and pricing details are from vendor/secondary posts (Business Wire, Prime.ai, AssemblyAI blog, independent research) and may have changed; verify current free-tier terms in the dashboard before relying on them.